

<u>Semantic HTML</u> <header><h1><nav><a><main> <h2><a><article><footer><p>	Relative path: find FCD (first common directory), then start 1 below it A is FCD, so b/c
<u>.card p vs .card > p</u> 1: selects p elements anywhere inside .card 2: selects only direct child p elements	<u>Padding vs margin</u> padding is inside border, margin is outside border
<u>Flexbox vs CSS grid</u> Flex for 1-dimension row/column Grid for 2-dimension row/column	@media: used for mobile view grid-template-columns: 1fr //1 col, 3, 1fr // 3 cols
<u>.button-row, button in row, vertical center, 12px gap</u> .button-row {display: flex; align-items: center; gap: 12px}	<u>.profile-card img full width</u> .profile-card img {width: 100%; max-width: 100; display: block;}
console.log(sizes[2]) // = size at index 2	<u>Map vs forEach:</u> Map returns new transformed array forEach runs code for each line item and returns undefined
<u>Function two ways:</u> function addTax(price) { return price * 1.07; } const addTax = (price) => price * 1.07;	<u>onClick, read input, display in paragraph</u> function showName() { const input = document.querySelector('#name-input'); const output = document.querySelector('#output'); output.textContent = input.value; } document.querySelector('#show-name').addEventListener('click', showName);
<u>Filter in-stock, map each to HTML string, joins strings, appends results to #results:</u> const html = products .filter(product => product.inStock) .map(product => `<article class="product"><h3>\${product.name}</h3><p>\${product.price}</p></article>`) .join("");	GET = retrieve POST = create/send PATCH = update part DELETE = remove
GET /api/comments = list comments POST /api/comments = create a comment PATCH /api/comments/42 = update comment 42 DELETE /api/comments/42 = delete comment 42	<u>App dev. w/specifications: (2resources, path, 2-4 fields, 1 queryable field)</u> Resource: CoffeeSpot Path: /api/coffee-spots Fields: name, neighborhood, rating, notes Queryable: name, neighborhood, rating Resource: Category Path: /api/categories Fields: name, description Queryable: name
Storing records in SQLite = backend Rendering cards on page = frontend Validating whether a bearer token is accepted = backend Showing a login form = frontend	<u>JWT?</u> JWT is used to prove user is authenticated <u>Passwords hashed before storage in database.</u> <u>Why?</u> So database does not directly contain readable passwords if exposed

Defining /api/comments as a REST resource = backend Storing the access token in browser = frontend	
JSX = JavaScript syntax that lets React describe UI with HTML-like tags	<u>props vs state:</u> props = inputs passed into component state = data a component remembers and can update
const [count, setCount] = useState (0); count starts at 0 setCount updates state React reloads when state changes	<u>What is wrong w/function w/siblings in return:</u> Return can only be one element, must have wrapper around sibling tags
<u>Key. Why?</u> Unique elements that React can track <u>useEffect is for fetching data after render</u>	Problems for react: if list/array, likely missing key Filter for grade > 90: const hStudent = students.filter(student => student.grade >= 90);
put output in return with `` and a __toHTML function: function courseToHTML(course) { return ` <article class="course"> <h3>\${course.title}</h3> <p>Room: \${course.room}</p> <p>Instructor: \${course.instructor.name}</p> </article> `; } }	<u>Fetch that creates comment:</u> await fetch('/api/comments', { method: 'POST', headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer \${token}` }, body: JSON.stringify({ post_id: 42, body: 'Great photo!' }) });
response.json needs await in front	<u>/api/sneakers = endpoint, name = queryable, search</u> <u>"running" in name</u> <u>/api/sneakers?name=running</u>
ORDER: -User logs in and receives token -Frontend stores token -React requests data with token in `Authorization` header -Backend verifies token, reads user-specific data from database, returns JSON -React stores response in state and renders list with `map`	<u>Export in react:</u> `export` makes component, function, or value available to import on another file. Aids in compartmentalization
Put names in list: (ul = unordered list) function StudentList({ students }) { return ({students.map(student => (<li key={student.id}>{student.name}))}); }	<u>React counter component. Starts at 0, onclick +1:</u> function Counter() { const [count, setCount] = useState(0); return (<button onClick={() => setCount(count + 1)}> {count} </button>); }